Implement parts-of-speech tagging using Sequence-to-Sequence architecture

To implement a Sequence-to-Sequence model that maps a sequence of English words to their corresponding parts-of-speech tags.

Algorithm
1. Import TensorFlow and the required Keras libraries.
2. Define sample English sentences and their corresponding POS tags.
3. Create separate tokenizers for input words and output tags.
4. Convert sentences and tags into integer sequences.
5. Pad all sequences to a fixed length.
6. Create an Embedding layer for input words.
7. Add an LSTM encoder to process the input sentence.
8. Use RepeatVector to pass the encoded representation to the decoder.
9. Add an LSTM decoder to generate the output sequence.
10. Add a TimeDistributed Dense layer with softmax activation.
11. Compile and train the model using the sentence and tag sequences.
12. Predict POS tags for a sample sentence and display the result.

Program

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
tf.keras.utils.set_random_seed(30)
# Sample sentences
sentences = [
"the cat eats fish",
"a dog likes meat",
"the boy plays cricket",
"a girl reads books",
"birds fly in sky",
"the teacher explains lessons",
"the student writes notes",
"a cat drinks milk",
"dogs chase balls",
"the child likes music"
]
# POS tags
tags = [
"DET NOUN VERB NOUN",
"DET NOUN VERB NOUN",
"DET NOUN VERB NOUN",
"DET NOUN VERB NOUN",
"NOUN VERB ADP NOUN",
"DET NOUN VERB NOUN",
"DET NOUN VERB NOUN",
"DET NOUN VERB NOUN",
"NOUN VERB NOUN",
"DET NOUN VERB NOUN"
]
# Tokenizers
word_tok = Tokenizer(filters="")
tag_tok = Tokenizer(
filters="",
lower=False
)
word_tok.fit_on_texts(sentences)
tag_tok.fit_on_texts(tags)
# Convert to sequences
X = pad_sequences(
word_tok.texts_to_sequences(sentences),
padding='post',
maxlen=4
)

Y = pad_sequences(
```

```
tag_tok.texts_to_sequences(tags),
padding='post',
maxlen=4
)
num_words = len(word_tok.word_index) + 1
num_tags = len(tag_tok.word_index) + 1
# Seq2Seq model
model = models.Sequential([
layers.Embedding(num_words, 32),
layers.LSTM(64),
layers.RepeatVector(4),
layers.LSTM(64, return_sequences=True),
layers.TimeDistributed(
layers.Dense(num_tags, activation='softmax')
)
])
# Compile
model.compile(
optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)
# Train
model.fit(
X, Y[..., None],
epochs=250,
verbose=0
)
# Predict
sample = "the cat eats fish"
sample_x = pad_sequences(
word_tok.texts_to_sequences([sample]),
maxlen=4,
padding='post'
)
pred = model.predict(
sample_x,

verbose=0
).argmax(axis=-1)[0]
id_to_tag = {
v: k for k, v in tag_tok.word_index.items()
}
print("Sentence:", sample)
print(
"Predicted POS Tags:",
[id_to_tag.get(int(i), "PAD") for i in pred]
)
```

```
Output of Program
Sentence: the cat eats fish
Predicted POS Tags:
['DET', 'NOUN', 'VERB', 'NOUN']
```

Result

Thus, a basic Sequence-to-Sequence model was successfully implemented for POS tagging. The model learned the relationship between English words and their corresponding grammatical tags.